

PROYECTO DE INGENIERÍA DE SOFTWARE

Plato Solidario

Plataforma tecnológica de logística social y distribución de raciones subsidiadas

Estefany Dayana Vela Rangel

C.C. 1193213518

Facultad de Ingenierías y Arquitectura

Programa de Ingeniería de Software

Universidad de Pamplona

2026

1. Introducción y Marco de Trabajo

1.1. Objetivos del Proyecto

Objetivo General

Diseñar y especificar una plataforma tecnológica integral (aplicación móvil y panel web) de logística social y distribución en tiempo real que intercepte y redistribuya raciones de almuerzo previamente adquiridas por estudiantes que no pueden reclamarlas un día determinado, entregándolas a estudiantes que ese día no cuentan con almuerzo, optimizando el aprovechamiento de recursos alimentarios.

Objetivos Específicos

- Garantizar una velocidad de procesamiento síncrono que permita la gestión completa de reservas y notificaciones push masivas en una ventana crítica estricta de 30 minutos (2:00 PM a 2:30 PM).
- Proveer un mecanismo transparente y auditable de trazabilidad mediante la generación y validación de códigos QR dinámicos con tiempo de expiración controlado.
- Asegurar el cumplimiento normativo estricto con la legislación colombiana de protección de datos (Ley 1581 de 2012), restringiendo el acceso exclusivamente a la comunidad estudiantil institucional validada.

1.2. Alcance del Proyecto

El proyecto abarca la definición de requerimientos, el modelado arquitectónico, el diseño detallado de interfaces y contratos de API, la estructuración de la base de datos y la planificación del control de calidad.

Queda explícitamente fuera del alcance:

- El desarrollo del código fuente de las aplicaciones móvil y web.
- La gestión de transacciones monetarias o pasarelas de pago, dado que el servicio opera sobre raciones previamente adquiridas y pagadas por otros estudiantes, que se redistribuyen sin costo adicional al estudiante beneficiario final.
- El soporte técnico físico de los dispositivos de escaneo en las instalaciones del comedor.

2. Requerimientos del Software (Documento SRS)

2.1. Requerimientos Funcionales (RF)

ID	Nombre	Descripción	Prioridad
RF-01	Autenticación OAuth2	El sistema debe restringir el acceso permitiendo el inicio de sesión únicamente a usuarios con cuentas de correo @unipamplona.edu.co.	Alta
RF-02	Liberación Automática de Inventario	El sistema debe activar el inventario disponible de raciones no reclamadas exactamente a las 2:00 PM de forma síncrona, calculando la diferencia entre almuerzos preparados y entregados en el sistema principal.	Alta
RF-03	Notificaciones Push Masivas	Al liberarse el inventario, el motor de mensajería debe enviar notificaciones push a todos los estudiantes registrados y activos en un tiempo menor a 10 segundos.	Alta
RF-04	Reserva bajo Política FCFS	Las reservas de raciones se asignarán bajo la política First-Come, First-Served. El sistema rechazará solicitudes cuando el inventario llegue a cero.	Alta
RF-05	Generación de QR Dinámico	Al confirmar la reserva, el sistema debe generar un código QR dinámico único vinculado al estudiante, con vigencia máxima de 15 minutos o hasta las 2:30 PM.	Alta
RF-06	Validación por Escaneo en Comedor	El panel web del operario debe permitir el escaneo del código QR, validar su estado (Activo, Expirado, Utilizado) y registrar la entrega en tiempo real.	Alta
RF-07	Tablero Analítico y de Impacto	El sistema administrativo debe consolidar reportes en tiempo real sobre raciones liberadas, efectividad de entrega, tiempos promedio e impacto en reducción de desperdicio.	Media

2.2. Requerimientos No Funcionales (RNF)

ID	Categoría	Descripción	Métrica
RNF-01	Rendimiento y Concurrencia	El backend debe procesar hasta 5,000 solicitudes simultáneas por segundo en	≤5ms por op.

ID	Categoría	Descripción	Métrica
		el minuto de apertura (2:00 PM) sin degradación del rendimiento.	
RNF-02	Disponibilidad Crítica	El sistema debe garantizar una disponibilidad del 99.9% durante la ventana operativa de 1:55 PM a 2:40 PM.	99.9% uptime
RNF-03	Seguridad y Privacidad	Todos los datos personales deben almacenarse encriptados en reposo (AES-256) y en tránsito (TLS 1.3), con consentimiento explícito de tratamiento de datos (Ley 1581/2012).	AES-256 / TLS 1.3
RNF-04	Restricción de Caducidad	Los tokens en los códigos QR deben invalidarse automáticamente en el servidor al expirar los 15 minutos de vigencia, impidiendo ataques de repetición.	TTL=15 min

2.3. Matriz de Trazabilidad de Requisitos

La siguiente tabla establece la correspondencia completa entre cada requisito, su caso de uso vinculado y los casos de prueba que verifican su cumplimiento. Se han completado los requisitos RF-03, RF-07 y todos los RNF que estaban ausentes en la versión inicial.

ID Requisito	Descripción	Caso de Uso	Casos de Prueba
RF-01	Autenticación con correo institucional	CU-01: Autenticar Estudiante	CP-01, CP-02
RF-02	Liberación automática 2:00 PM	CU-02: Liberar Inventario	CP-03
RF-03	Notificaciones push masivas (<10s)	CU-02: Liberar Inventario	CP-03B
RF-04	Reserva FCFS en tiempo real	CU-03: Reservar Almuerzo	CP-04, CP-05
RF-05	Generación de QR dinámico (15 min)	CU-03: Reservar Almuerzo	CP-06
RF-06	Validación por escaneo de QR	CU-04: Validar Entrega QR	CP-07, CP-08
RF-07	Tablero analítico y de impacto	CU-05: Consultar Analíticas	CP-09
RNF-01	Concurrencia 5.000 req/s	CU-03: Reservar Almuerzo	CP-LR-01 (Carga)

ID Requisito	Descripción	Caso de Uso	Casos de Prueba
RNF-02	Disponibilidad 99.9%	Todos los CU críticos	CP-HA-01 (Disponibilidad)
RNF-03	Encriptación AES-256 / TLS 1.3	CU-01: Autenticar Estudiante	CP-SEC-01
RNF-04	Caducidad automática de tokens	CU-04: Validar Entrega QR	CP-07

2.4. Modelos de Casos de Uso

2.4.1. Diagrama de Casos de Uso (Descripción Textual del Modelo)

El sistema Plato Solidario cuenta con tres actores principales y cinco casos de uso nucleares que cubren el ciclo completo de redistribución de raciones:

Actores del Sistema:

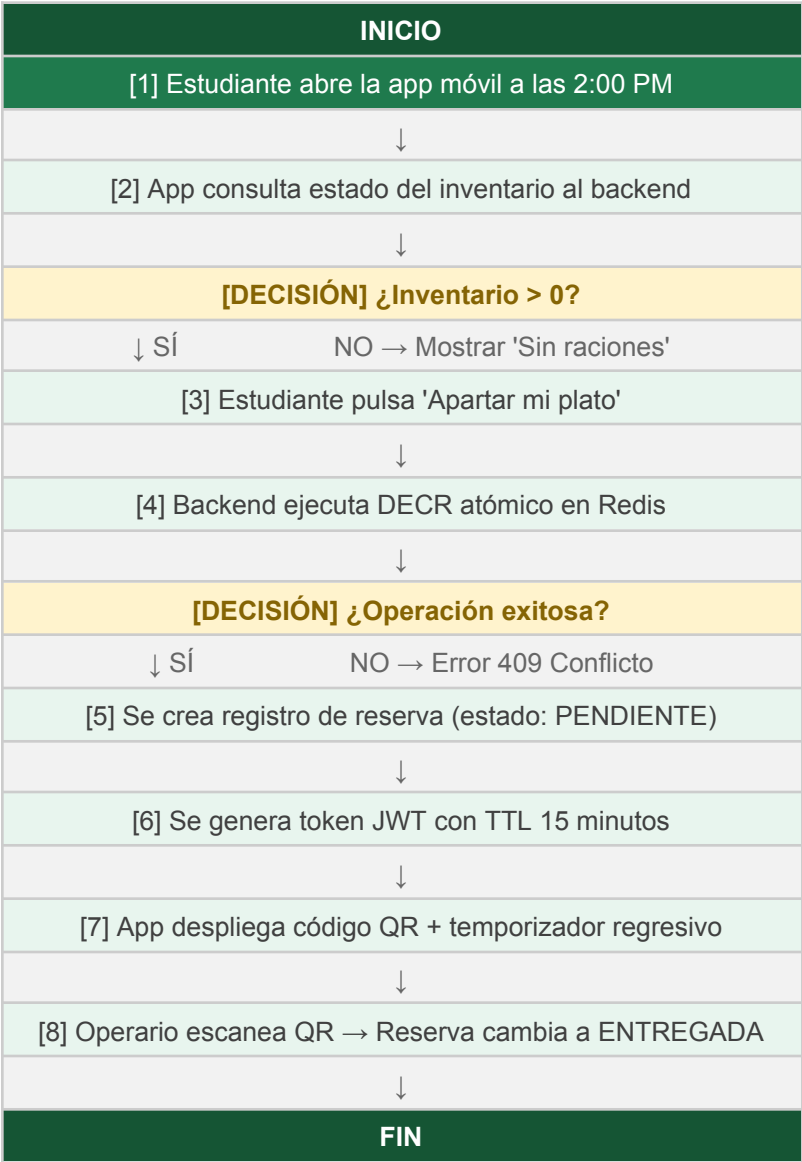
- Estudiante Beneficiario: Usuario final que consulta disponibilidad, realiza reservas y presenta el código QR en el comedor.
- Operario del Comedor: Personal encargado de escanear y validar los códigos QR para registrar la entrega de la ración.
- Sistema Administrador: Módulo automatizado responsable de ejecutar la liberación de inventario a las 2:00 PM y disparar las notificaciones masivas.

Casos de Uso del Sistema:

ID	Caso de Uso	Actor	Descripción
CU-01	Autenticar Estudiante	Estudiante	Ingreso al sistema con correo @unipamplona.edu.co mediante OAuth2 institucional.
CU-02	Liberar Inventario	Sistema Admin	Activación automática del inventario de raciones sobrantes a las 2:00 PM con notificaciones push.
CU-03	Reservar Almuerzo	Estudiante	Selección y reserva de una ración disponible bajo política FCFS con generación de QR.
CU-04	Validar Entrega QR	Operario Comedor	Escaneo y verificación del código QR del estudiante para registrar la entrega.
CU-05	Consultar Analíticas	Sistema Admin	Visualización de métricas de impacto, raciones entregadas y tiempos de recolección.

2.4.2. Diagrama de Flujo — Proceso de Reserva (CU-03)

El siguiente flujo describe la secuencia de pasos en el proceso principal de reserva de ración solidaria:



2.4.3. Caso de Uso CU-03: Reservar Almuerzo (Detallado)

Actor Principal: Estudiante Beneficiario.

Precondiciones:

- El estudiante ha iniciado sesión con su cuenta institucional validada.
- El sistema se encuentra dentro de la ventana de 2:00 PM a 2:30 PM.
- El inventario disponible de raciones es mayor a cero.

Flujo Principal:

1. El estudiante recibe la notificación push de raciones disponibles o ingresa directamente a la aplicación móvil a las 2:00 PM.
2. La aplicación solicita el estado del inventario síncrono al backend vía API REST.
3. El estudiante hace clic en el botón 'Apartar mi plato'.
4. El backend procesa la transacción de forma atómica disminuyendo en 1 la cantidad disponible en la caché de Redis.
5. El backend genera un registro de reserva en estado PENDIENTE y crea un token JWT dinámico con expiración fija de 15 minutos.
6. La aplicación móvil despliega en pantalla el código QR dinámico junto a un temporizador de cuenta regresiva.

Flujo Alternativo A — Inventario Agotado:

En el paso 4, si el inventario llega a cero concurrentemente, el sistema bloquea la transacción, devuelve HTTP 409 Conflict y la aplicación muestra el mensaje: 'Las raciones del día se han terminado'.

Postcondiciones:

- Se ha creado un registro en la tabla reservas con estado PENDIENTE.
- El contador de Redis se ha decrementado en una unidad.
- El estudiante tiene activo un código QR válido por 15 minutos.

3. Arquitectura de Software (Documento SAD)

3.1. Diagrama de Contenedores — Modelo C4

Para responder a la necesidad crítica de concurrencia y velocidad en tiempo real, el sistema adopta un modelo de arquitectura desacoplado basado en los siguientes contenedores lógicos:

Contenedor	Descripción y Responsabilidad
 App Móvil (iOS/Android)	Interfaz nativa para el estudiante. Recibe notificaciones Firebase Cloud Messaging (FCM), renderiza códigos QR a partir de tokens JWT firmados y muestra el temporizador regresivo de vigencia.
 Panel Web SPA (Admin/Comedor)	Single Page Application para operarios y administradores. Permite el escaneo de QR mediante cámara web y visualización de analíticas agregadas en tiempo real.
 API Gateway / Backend	Servicio asíncrono no bloqueante (Node.js/Fastify o Go). Centraliza la seguridad, autenticación OAuth2 institucional, validación de negocio y orquestación de servicios.
 Caché In-Memory (Redis)	Contenedor crítico del sistema. Almacena de forma efímera y atómica el contador de raciones disponibles desde las 2:00 PM. Garantiza operaciones concurrentes en microsegundos mediante DECR atómico y gestiona la caducidad nativa (TTL) de las reservas.
 Base de Datos Relacional (PostgreSQL)	Almacén persistente principal. Guarda información histórica de usuarios, perfiles, raciones por fecha, auditorías de entregas y datos maestros. Se sincroniza de forma asíncrona tras confirmación en Redis.
 Servicio de Mensajería (FCM)	Componente externo de Firebase Cloud Messaging. Recibe las instrucciones del backend para el envío masivo de notificaciones push a todos los dispositivos suscritos en menos de 10 segundos.

3.2. Registro de Decisiones de Arquitectura (ADR)

ADR-001: Adopción de Redis para el Manejo de Inventario de Alta Concurrencia

Estado	☒ Aprobado
Fecha	Junio 2026
Contexto	A las 2:00 PM, miles de estudiantes intentarán reservar de forma simultánea un número limitado de almuerzos sobrantes (aprox. 150 raciones). El uso directo de consultas en una BD relacional generaría bloqueos de filas (row locks), tiempos de espera elevados y condiciones de carrera (race conditions).
Decisión	Utilizar operaciones atómicas de decremento (DECR) en Redis para el control del inventario en tiempo real. La base de datos persistente solo se actualizará de forma asíncrona una vez confirmada la reserva en la capa in-memory.

Consecuencias	Reducción drástica del tiempo de respuesta de ~200ms a <5ms. Mitigación absoluta de sobreventa de almuerzos. Requiere lógica de conciliación al finalizar la jornada para sincronizar estados entre Redis y PostgreSQL.
Alternativas Descartadas	(1) PostgreSQL con SELECT FOR UPDATE: descartado por riesgo de deadlocks a alta concurrencia. (2) Cola de mensajes (RabbitMQ/Kafka): añade latencia innecesaria para una operación que debe resolverse en milisegundos.

ADR-002: Autenticación OAuth2 con Dominio Institucional

Estado	✅ Aprobado
Contexto	El servicio está restringido exclusivamente a la comunidad estudiantil de la Universidad de Pamplona. Se requiere validación institucional sin gestionar credenciales propias.
Decisión	Implementar OAuth2 delegando la autenticación al proveedor de identidad institucional (@unipamplona.edu.co). Los tokens de sesión son JWT firmados con clave asimétrica RS256.
Consecuencias	Se elimina la gestión de contraseñas propias. El sistema hereda la seguridad del IdP institucional. Los estudiantes sin cuenta institucional activa quedan automáticamente excluidos.

4. Diseño del Software

4.1. Diagrama de Clases Conceptual

La siguiente tabla describe la estructura del dominio lógico del sistema con sus clases, atributos, métodos y relaciones:

Clase	Atributos	Métodos	Relación
Usuario (Clase Base)	id: UUID nombre: String email: String rol: Enum	autenticar()	Base de Estudiante y Operario
Estudiante (Hereda a Usuario)	codigoEstudiante: String estadoSancion: Boolean	solicitarReserva() cancelarReserva()	Hereda de Usuario Tiene → Reserva (1..*)
OperarioComedor (Hereda Usuario)	idComedor: Integer	escanearCodigoQR() registrarEntregaManual()	Hereda de Usuario Valida → TicketQR
RacionAlimento	idRacion: UUID fecha: Date cantidadInicial: Integer cantidadDisponible: Integer	liberarInventario() decrementarCantidad()	Tiene → Reserva (1..*)
Reserva	idReserva: UUID estudianteId: UUID racionId: UUID horaCreacion: Timestamp estado: Enum	confirmarEntrega() invalidarPorTiempo()	Pertenece a → Estudiante Referencia → RacionAlimento Genera → TicketQR (1..1)
TicketQR	tokenJWT: String fechaExpiracion: Timestamp	generarToken() validarFirma()	Pertenece a → Reserva (1..1)

Relaciones del Diagrama de Clases:

- Usuario ◀|— Estudiante (Herencia / Generalización)
- Usuario ◀|— OperarioComedor (Herencia / Generalización)
- Estudiante —< Reserva (Asociación 1 a muchos: un estudiante puede tener múltiples reservas históricas, pero solo una activa por día)
- RacionAlimento —< Reserva (Asociación 1 a muchos: una ración diaria puede tener múltiples reservas)
- Reserva — TicketQR (Composición 1 a 1: el ticket QR es parte integral de la reserva y no existe sin ella)
- OperarioComedor —> TicketQR (Dependencia: el operario valida el ticket QR)

4.2. Contratos de API (Endpoints del Sistema)

Endpoint 1: Creación de Reserva

POST /api/v1/reservas

Descripción: Crea una reserva atómica para el estudiante autenticado bajo la política FCFS.

Request Headers:

```
Authorization: Bearer <JWT_TOKEN_INSTITUCIONAL>
Content-Type: application/json
```

Request Body:

```
{
  "racion_id": "8f3b9c4d-2a1b-4c6d-8e7f-0a1b2c3d4e5f"
}
```

Response 201 Created:

```
{
  "reserva_id": "acbd1234-fa43-4321-9876-0987654321ba",
  "estado": "PENDIENTE",
  "hora_limite_recogida": "2026-06-15T14:15:00Z",
  "qr_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Response 409 Conflict — Inventario Agotado:

```
{
  "codigo_error": "INVENTARIO_AGOTADO",
  "mensaje": "No quedan raciones disponibles para reserva en la jornada actual."
}
```

Endpoint 2: Validación de Entrada QR

PATCH /api/v1/reservas/validar

Descripción: Valida el código QR presentado por el estudiante y registra la entrega de la ración.

Request Headers:

```
Authorization: Bearer <JWT_TOKEN_OPERARIO>
Content-Type: application/json
```

Response 200 OK:

```
{
  "resultado": "VALIDADO_EXITOSO",
  "mensaje": "Entrega de ración registrada para el estudiante.",
  "estudiante": "Jaime Mateo Ruiz Buitrago",
  "hora_entrega": "2026-06-15T14:07:23Z"
}
```

Response 400 Bad Request — Ticket Expirado:

```
{
  "resultado": "VALIDACION_FALLIDA",
  "codigo_error": "TICKET_EXPIRADO",
}
```

```

"mensaje": "El código QR presentado superó el límite de vigencia de 15 minutos."
}

```

Endpoint 3: Consulta de Inventario en Tiempo Real

GET /api/v1/raciones/disponibles

Descripción: Retorna el estado actual del inventario de raciones para la jornada activa. Utilizado por la app móvil para actualizar el contador en pantalla.

Response 200 OK:

```

{
  "fecha": "2026-06-15",
  "ventana_activa": true,
  "raciones_disponibles": 47,
  "hora_cierre": "2026-06-15T14:30:00Z"
}

```

4.3. Prototipos UX/UI (Especificación de Flujo)

Pantalla	Elementos UI	Comportamiento / UX
Inicio (Estudiante)	— Header con logo institucional y nombre del usuario.— Indicador de estado ('Cerrado' / '¡Disponible!').— Contador gigante de raciones disponibles.— Botón CTA: 'Apartar mi plato' (deshabilitado fuera de ventana).— Paleta: gris neutro + verde institucional.	— Estado 'Cerrado' antes de las 2:00 PM: botón en gris inactivo.— A las 2:00 PM: cambio animado a estado activo, notificación push.— Contador se actualiza en tiempo real vía WebSocket.— Al agotarse stock: botón desaparece, aparece mensaje de aviso.
Ticket de Reserva (QR)	— Fondo de color sólido verde institucional.— Código QR central de alta densidad y tamaño máximo.— Temporizador regresivo MM:SS en fuente grande.— Datos del estudiante y número de ración asignado.	— Temporizador decrementa en tiempo real desde 15:00.— A los 2 minutos restantes: fondo cambia a naranja (alerta).— Al llegar a 00:00: QR se oscurece con overlay 'EXPIRADO'.— El QR no se puede capturar por pantalla (bloqueo de captura).
Validación Operario (Web)	— Barra lateral con accesos a escaneo y analíticas.— Área central con visor de cámara para escaneo.— Panel lateral con historial de entregas del día.— Indicadores de color: verde (válido), rojo (expirado/usado).	— Al detectar QR: procesamiento automático sin botón de confirmar.— Feedback inmediato: sonido + pantalla verde (éxito) o roja (error).— Registro automático de hora de entrega en la tabla reservas.
Tablero Analítico (Admin)	— Tarjetas KPI: raciones liberadas, entregadas, expiradas.— Gráfico de barras: entregas por franja horaria.— Mapa de calor: densidad de retiro por semana.— Exportación a CSV/Excel.	— Datos actualizados en tiempo real durante la ventana operativa.— Resumen diario disponible a partir de las 3:00 PM.— Filtros por fecha, semana y programa académico.

5. Construcción de Software

5.1. Script de Base de Datos (DDL Relacional)

Script de creación de la estructura relacional del sistema Plato Solidario en PostgreSQL 15+:

```
-- =====
-- DDL: Plato Solidario – PostgreSQL 15+
-- Universidad de Pamplona, 2026
-- =====

-- Tipos enumerados del dominio
CREATE TYPE rol_usuario AS ENUM ('ESTUDIANTE', 'OPERARIO', 'ADMINISTRADOR');
CREATE TYPE estado_reserva AS ENUM ('PENDIENTE', 'ENTREGADA', 'EXPIRADA',
'CANCELADA');

-- Tabla de usuarios institucionales
CREATE TABLE usuarios (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre_completo   VARCHAR(150) NOT NULL,
    email             VARCHAR(100) UNIQUE NOT NULL
                        CHECK (email LIKE '%@unipamplona.edu.co'),
    rol               rol_usuario NOT NULL DEFAULT 'ESTUDIANTE',
    fecha_registro    TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    activo            BOOLEAN NOT NULL DEFAULT TRUE
);

-- Tabla de raciones diarias
CREATE TABLE raciones_diarias (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    fecha             DATE UNIQUE NOT NULL DEFAULT CURRENT_DATE,
    cantidad_inicial  INT NOT NULL CHECK (cantidad_inicial >= 0),
    cantidad_disponible INT NOT NULL CHECK (cantidad_disponible >= 0),
    hora_liberacion    TIMESTAMP WITH TIME ZONE,
    hora_cierre        TIMESTAMP WITH TIME ZONE
);

-- Tabla principal de reservas
CREATE TABLE reservas (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    usuario_id        UUID NOT NULL REFERENCES usuarios(id) ON DELETE RESTRICT,
    racion_id         UUID NOT NULL REFERENCES raciones_diarias(id) ON DELETE
RESTRICT,
    hora_reserva       TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    hora_limite        TIMESTAMP WITH TIME ZONE NOT NULL,
    estado            estado_reserva NOT NULL DEFAULT 'PENDIENTE',
    token_verificacion TEXT UNIQUE NOT NULL
);

-- Índices optimizados para la ventana crítica de búsqueda
CREATE INDEX idx_reservas_usuario_fecha ON reservas(usuario_id, hora_reserva);
CREATE INDEX idx_reservas_estado        ON reservas(estado) WHERE estado =
'PENDIENTE';
CREATE INDEX idx_raciones_fecha          ON raciones_diarias(fecha);
```

5.2. Diccionario de Datos Detallado

Tabla: usuarios

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	NO	PK	Identificador único autogenerado para cada usuario del sistema.
nombre_completo	VARCHAR(150)	NO	—	Nombre legal completo del usuario.
email	VARCHAR(100)	NO	UK	Correo institucional. Restricción: debe terminar en @unipamplona.edu.co.
rol	ENUM	NO	—	Rol del usuario en el sistema: ESTUDIANTE, OPERARIO o ADMINISTRADOR.
fecha_registro	TIMESTAMP TZ	NO	—	Marca de tiempo de creación del registro.
activo	BOOLEAN	NO	—	Indica si la cuenta está habilitada para operar en el sistema.

Tabla: raciones_diarias

Columna	Tipo	Nulo	Clave	Descripción
id	UUID	NO	PK	Identificador único de la tanda de raciones de un día específico.
fecha	DATE	NO	UK	Fecha de la jornada. Restricción UNIQUE: una sola tanda por día.
cantidad_inicial	INTEGER	NO	—	Total de raciones preparadas por el comedor al inicio del día.
cantidad_disponible	INTEGER	NO	—	Raciones sobrantes no reclamadas, disponibles para redistribución.
hora_liberacion	TIMESTAMP TZ	SÍ	—	Momento exacto en que el inventario fue activado (aprox. 2:00 PM).
hora_cierre	TIMESTAMP TZ	SÍ	—	Momento en que se cerró la ventana de reservas (aprox. 2:30 PM).

Tabla: reservas

Columna	Tipo	Nulo	Clave	Descripción / Restricciones
id	UUID	NO	PK	Identificador único autogenerado para cada reserva del sistema.

Columna	Tipo	Nulo	Clave	Descripción / Restricciones
usuario_id	UUID	NO	FK	Relación con el registro del estudiante solicitante (ON DELETE RESTRICT).
racon_id	UUID	NO	FK	Vínculo a la tanda de raciones del día correspondiente (ON DELETE RESTRICT).
hora_reserva	TIMESTAMP TZ	NO	—	Marca de tiempo exacta de asignación de la reserva.
hora_limite	TIMESTAMP TZ	NO	—	Momento de expiración. Calculada como hora_reserva + 15 minutos (no mayor a las 14:30).
estado	ENUM	NO	—	Ciclo de vida del ticket: PENDIENTE, ENTREGADA, EXPIRADA, CANCELADA.
token_verificacion	TEXT	NO	UK	Token JWT firmado RS256. Restricción UNIQUE: un token por reserva.

5.3. Documentación del Desarrollador (Guía Técnica)

Requisitos del Entorno de Desarrollo

Herramienta	Versión Mínima	Propósito
Docker Desktop	v24+	Orquestación de servicios locales (Redis, PostgreSQL, API).
Node.js LTS	v20+	Runtime del servidor API (Fastify) y herramientas de build.
PostgreSQL	v15+	Base de datos relacional principal del sistema.
Redis Server	v7.2+	Caché in-memory para control de inventario concurrente.
Git	v2.40+	Control de versiones del repositorio de código fuente.

Estructura Base del Repositorio Backend

```
/plato-solidario-api
├── /src
│   ├── /config           # Configuraciones: Redis, DB, Seguridad JWT
│   ├── /controllers      # Controladores HTTP: Auth, Reservas, Analíticas
│   ├── /domain           # Modelos de dominio puro y lógica de negocio
│   ├── /jobs             # Tareas cron (expiración automática 2:30 PM)
│   ├── /repositories     # Capa de abstracción de acceso a datos (Data Mappers)
│   ├── /middleware       # Auth middleware, rate limiting, validación de esquemas
│   └── app.js            # Punto de entrada de la aplicación
```



```
|— /tests
|   |— /unit          # Pruebas unitarias (Jest)
|   |— /integration   # Pruebas de integración (Supertest + BD efímera)
|   |— /load          # Scripts de carga (K6 / Apache JMeter)
|— docker-compose.yml  # Despliegue local controlado de servicios auxiliares
|— .env.example        # Variables de entorno requeridas
|— package.json
```

6. Pruebas de Software

6.1. Plan de Pruebas — Estrategia, Alcance y Criterios de Aceptación

Estrategia General

Dada la criticidad del factor tiempo (ventana síncrona de 30 minutos) y la naturaleza concurrente del sistema, el aseguramiento de la calidad se estructura en tres niveles estrictos, ejecutados de forma secuencial y obligatoria antes de cualquier despliegue en producción:

Nivel	Tipo de Prueba	Descripción
Nivel 1	Pruebas Unitarias Aisladas	Cobertura exhaustiva sobre la lógica de generación y firma del token JWT, validación de zonas horarias locales y cálculo de expiración de QR.
Nivel 2	Pruebas de Integración	Validación de la persistencia correcta y el decremento exacto en Redis al simular solicitudes simultáneas bajo la política FCFS. Incluye pruebas de sincronización Redis ↔ PostgreSQL.
Nivel 3	Pruebas de Carga y Estrés Crítico	Ejecución de scripts de carga con K6 o Apache JMeter para replicar la entrada concurrente masiva de 5.000 usuarios exactamente a las 2:00:00 PM, midiendo tasas de error y tiempos de respuesta.

Alcance de las Pruebas

Módulos incluidos en el plan de pruebas:

- Módulo de Autenticación OAuth2 (RF-01)
- Motor de Liberación de Inventario y Notificaciones (RF-02, RF-03)
- Motor de Reservas FCFS con Redis (RF-04)
- Generador y Validador de Códigos QR / JWT (RF-05, RF-06)
- Tablero Analítico (RF-07)
- Capa de Seguridad y Encriptación (RNF-03, RNF-04)

Fuera del alcance de las pruebas:

- Pruebas de usabilidad con usuarios reales (corresponden a una fase posterior de UX Research).
- Pruebas del dispositivo físico de escaneo en instalaciones del comedor.
- Pruebas de integración con el sistema de nómina de la universidad.

Criterios de Aceptación

Criterio	Condición de Aceptación
Cobertura mínima	No se aprobará ningún PR si la cobertura total del proyecto cae por debajo del 85%.
Tasa de errores en carga	La tasa de error bajo carga de 5.000 usuarios concurrentes no debe superar el 0.1%.
Tiempo de respuesta P95	El percentil 95 del tiempo de respuesta del endpoint POST /reservas debe ser < 50ms.
Integridad de stock	Bajo ninguna circunstancia el sistema puede entregar más raciones de las disponibles (inventario nunca negativo).
Caducidad de tokens	El 100% de los tokens QR deben invalidarse correctamente al alcanzar su TTL de 15 minutos.
Autenticación	El 100% de los accesos con correo no institucional deben ser rechazados con HTTP 403.

6.2. Casos de Prueba — Guiones Detallados con Resultados Esperados

ID	Componente	Escenario	Datos de Entrada	Resultado Esperado
CP-01	Módulo Auth	Autenticación exitosa con correo institucional válido.	Email: j.mateo@unipamplon a.edu.co	HTTP 200 OK. Emisión de JWT de sesión válido.
CP-02	Módulo Auth	Bloqueo de dominios comerciales externos.	Email: usuario@gmail.com	HTTP 403 Forbidden. Sin emisión de token.
CP-03	Motor Inventario	Liberación automática de raciones a las 2:00 PM exactas.	Hora del sistema: 14:00:00	Redis inicializado con cantidad_disponible. Evento disparado. hora_liberacion registrada en BD.
CP-03 B	Motor Notif.	Envío masivo de push en tiempo < 10 segundos.	1.000 dispositivos suscritos activos.	El 100% de los dispositivos reciben notificación en ≤ 10 segundos. Log de confirmación FCM registrado.
CP-04	Motor Reservas	Reserva exitosa con stock positivo disponible.	Stock Redis = 10. racion_id válido.	HTTP 201 Created. Stock Redis pasa a 9. Reserva con estado PENDIENTE creada en BD.
CP-05	Motor Reservas	Control estricto de condición de carrera (conurrencia).	Stock Redis = 1. Dos peticiones simultáneas en el mismo milisegundo.	Petición A: HTTP 201 (Stock Redis = 0). Petición B: HTTP 409

ID	Componente	Escenario	Datos de Entrada	Resultado Esperado
				Conflict. Stock Redis nunca llega a -1.
CP-06	Módulo QR	Generación de QR con expiración correcta.	Reserva confirmada a las 14:02:15.	Token JWT generado con exp = 14:17:15. QR renderizado en la app con temporizador activo.
CP-07	Módulo QR Scan	Validación de QR expirado superado el tiempo límite.	Escaneo a las 2:16 PM de un ticket creado a las 2:00 PM.	HTTP 400. Código TICKET_EXPIRADO. Estado de reserva cambia a EXPIRADA en BD.
CP-08	Módulo QR Scan	Intento de reutilización de un QR ya utilizado (anti-fraude).	Token de una reserva con estado ENTREGADA.	HTTP 400. Código TICKET_YA_UTILIZADO. No se modifica ningún registro.
CP-09	Tablero Admin	Visualización de analíticas del día en tiempo real.	Fecha con al menos 10 reservas en estados mixtos.	Dashboard muestra KPIs actualizados: raciones liberadas, entregadas, expiradas y tasa de éxito.
CP-LR-01	Carga (K6)	Estrés con 5.000 usuarios concurrentes a las 2:00 PM.	5.000 requests simultáneos. Stock inicial = 150.	Exactamente 150 reservas exitosas (HTTP 201). 4.850 rechazos (HTTP 409). Tasa error < 0.1%. P95 < 50ms.
CP-HA-01	Disponibilidad	Sistema disponible durante la ventana crítica completa.	Monitoreo continuo de 1:55 PM a 2:40 PM.	Uptime del 100% durante la ventana. Sin interrupciones del servicio API.
CP-SE-C-01	Seguridad	Datos en tránsito protegidos con TLS 1.3.	Inspección del handshake SSL con Wireshark/curl.	Conexión establece TLS 1.3. Datos ilegibles en texto plano. Certificado válido y no caducado.

6.3. Reporte de Cobertura — Porcentaje de Código Evaluado

El proyecto exige estándares mínimos de cobertura de código evaluados mediante herramientas de integración continua (Jest con Istanbul). Los umbrales deben cumplirse en cada build antes de permitir el merge a la rama principal:

Capa del Sistema	Cobertura Líneas	Cobertura Ramas	Herramienta
Controladores y Enrutamiento (API Gateway)	≥ 80%	≥ 75%	Jest + Supertest

Capa del Sistema	Cobertura Líneas	Cobertura Ramas	Herramienta
Servicios de Negocio (Lógica FCFS y QR Core)	≥ 95%	≥ 95%	Jest (Branch Coverage)
Acceso a Datos e Infraestructura	≥ 85%	≥ 80%	Jest + BD efímera Docker
Módulo de Autenticación OAuth2	≥ 90%	≥ 90%	Jest + mock IdP
Jobs Cron (expiración automática)	≥ 85%	≥ 85%	Jest con fake timers

Configuración del umbral en Jest (package.json):

```
"jest": {
  "coverageThreshold": {
    "global": {
      "branches": 85,
      "lines": 85,
      "functions": 85,
      "statements": 85
    },
    "./src/domain/": {
      "branches": 95,
      "lines": 95
    }
  }
}
```

7. Bibliografía y Referencias

- Brown, S. (2018). *Software architecture for developers: Visualise, document and explore your software architecture*. Leanpub.
- Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial n.º 48.587. Imprenta Nacional de Colombia.
- Fielding, R., & Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and content* (RFC 7231). Internet Engineering Task Force. <https://tools.ietf.org/html/rfc7231>
- Fowler, M. (2012). *Patterns of enterprise application architecture*. Addison-Wesley Professional.
- Institute of Electrical and Electronics Engineers. (2008). *IEEE standard for software and system test documentation* (IEEE Std 829-2008). <https://doi.org/10.1109/IEEESTD.2008.4578383>
- Jones, M. B., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://tools.ietf.org/html/rfc7519>
- Redis Ltd. (2023). *Redis documentation*. <https://redis.io/docs/>
- The PostgreSQL Global Development Group. (2023). *PostgreSQL 15 documentation*. <https://www.postgresql.org/docs/15/>